

Guía de Comandos Git para Estudiantes

CONTROL DE VERSIONES EN PROYECTOS ACADÉMICOS E INGENIERÍA DE SOFTWARE

Esta guía proporciona un resumen estructurado y práctico de los comandos esenciales de Git, organizados de acuerdo a su flujo de trabajo lógico. Es una herramienta ideal para el desarrollo de proyectos técnicos, facilitando tanto el aprendizaje individual como la colaboración en equipos bajo metodologías ágiles.

1. Configuración Inicial del Entorno

Antes de comenzar a registrar cambios en tus proyectos, es fundamental establecer tu identidad global dentro del sistema de control de versiones.

```
# Configura tu nombre completo en el sistema
git config --global user.name "Tu Nombre"

# Configura el correo asociado a tu cuenta de GitHub/GitLab
git config --global user.email "tu_correo@ejemplo.com"

# Verifica las configuraciones activas en el entorno
git config --list
```

2. Creación y Clonación de Repositorios

Comandos principales para iniciar un espacio de trabajo controlado de forma local o para unirse a proyectos remotos ya existentes.

Comando	Descripción	Caso de Uso Académico
<code>git init</code>	Inicializa un repositorio Git local vacío en la carpeta actual.	Al iniciar un proyecto de software o investigación desde cero.
<code>git clone <url></code>	Descarga una copia completa de un repositorio remoto junto con todo su historial.	Al integrarte al repositorio grupal compartido por tus compañeros.

3. El Ciclo de Vida de los Archivos (Flujo Local)

Git gestiona los archivos locales a través de tres estados principales: *Modified* (Modificado), *Staged* (Preparado para confirmar) y *Committed* (Confirmado en el historial).

```
# Muestra el estado del directorio de trabajo y los archivos modificados
git status

# Añade un archivo específico al área de preparación (Staging Area)
git add nombre_del_archivo.py

# Añade todas las modificaciones y archivos nuevos de la carpeta actual
git add .

# Guarda de forma permanente los cambios preparados en el historial local
git commit -m "Mensaje descriptivo del cambio en tiempo presente"

# Permite modificar el mensaje del último commit local realizado
git commit --amend -m "Nuevo mensaje corregido"
```

4. Gestión de Ramas (Branches)

El uso de ramas independientes es vital en proyectos colaborativos. Evita alterar la estabilidad del código principal (`main`) mientras desarrollas nuevas funcionalidades de forma aislada.

Comando	Propósito y Operación
<code>git branch</code>	Lista todas las ramas locales. La rama activa se resalta con un asterisco (*).
<code>git branch <nombre></code>	Crea una nueva rama a partir de la rama actual, sin cambiarse a ella.
<code>git checkout <nombre></code>	Cambia el entorno de trabajo a la rama especificada.
<code>git checkout -b <nombre></code>	Atajo para crear una nueva rama e inmediatamente cambiarse a ella.
<code>git merge <nombre></code>	Fusiona la rama especificada dentro de la rama que tienes activa actualmente.
<code>git branch -d <nombre></code>	Elimina de forma lógica una rama local (usar solo tras fusionar sus cambios).

5. Sincronización con Repositorios Remotos

Permite la interacción constante y la subida de avances hacia servidores en la nube como GitHub, GitLab o Bitbucket.

```
# Vincula el repositorio local con un servidor remoto por primera vez
git remote add origin https://github.com/usuario/proyecto.git

# Envía tus commits locales a la rama remota correspondiente
git push -u origin nombre_de_tu_rama

# Descarga e integra automáticamente los cambios del servidor remoto en tu rama local
git pull

# Descarga la información del servidor remoto pero sin alterar tu código actual
git fetch
```

6. Historial y Diagnóstico

```
# Muestra el historial completo de commits de forma cronológica invertida
git log

# Muestra el historial en un formato compacto de una sola línea por cambio
git log --oneline

# Compara visualmente las diferencias de código no preparadas frente al último commit
git diff
```

7. Buenas Prácticas Recomendadas

- **Mensajes de Commit Atómicos:** Cada commit debe encargarse de resolver un solo problema o funcionalidad. Evita hacer commits masivos con mensajes ambiguos como "*Cambios finales*". Usa verbos en presente (Ej: "Configurar conexion a base de datos").
- **Configuración Obligatoria de `.gitignore`:** Antes de guardar tu primer commit, añade un archivo `.gitignore` en la raíz. Excluye explícitamente entornos virtuales (`venv/`), configuraciones de IDEs, archivos del sistema operativo y credenciales de APIs.
- **Sincronización Diaria:** Al iniciar tu jornada de desarrollo, cámbiate a la rama principal y realiza un `git pull`. Esto reduce drásticamente la probabilidad de enfrentarse a conflictos complejos de código.
- **Resolución de Conflictos:** Si Git detiene una fusión por conflictos de líneas, edita los archivos afectados con tu editor de código (como VS Code), decide junto a tu equipo qué líneas prevalecen, guarda el archivo, ejecuta `git add .` y concluye mediante un `git commit`.